
WoC

Machine Learning Bootcamp

Sai Sriram Dodda

23JE0322

Btech.Computer Science and Engineering



Report of Machine Learning Project

WOC-6.0

Problem Statement:

- Creating and Implementing machine learning algorithms from scratch using only Python, NumPy, Pandas and Matplotlib.
- Training the above models based on the given train data set and implementing on the test data set to check performance of the model.

NumPy- For performing Calculations as required.

Pandas- For handling huge data and processing.

Matplotlib-For data visualization, analysis of performance.



I have Implemented the following algorithms in order:

- Linear Regression
- Polynomial Regression
- Logistic regression
- KNN
- N-Layer Neural Network
- K means Clustering

As mentioned in proposal of project. I strictly followed the Week Wise Timeline mentioned:

Week 1:

• I set up the project environment in Google Collab.

• I implemented Linear Regression with NumPy, Pandas and Matplotlib.

• I was determined to learn KNN, K-Means Clustering, and Neural Networks, aiming to better grasp these concepts for my project. The goal was to improve my skills and apply these algorithms more effectively in the ongoing work.

Week 2:

• I extended Linear Regression to Polynomial Regression and developed this model.

• Then, I implemented Logistic Regression with NumPy.

Week 3:

• I implemented the KNN algorithm from scratch.

• I started coding N-layer Neural Networks.

Week 4:

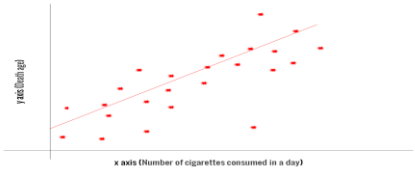
• I completed N-layer Neural Networks and also completed K-means Clustering.

• I trained and tested all algorithms on provided datasets and completed the tuning of hyperparameters.

• I created visualizations and training logs and completed the implementation of regularization and optimization methods.

• Finally, I began drafting the project report.

HERE IS THE BRIEF VIEW OF MY IMPLEMENTATION OF MACHINE LEARNING ALGORTHMS.....



1. LINEAR REGRESSION:

- In simpler terms, it helps us understand and predict the behavior of a variable based on its relationship with other variables. In the project given, where the training data consists of 50,000 observations with multiple features and the test data has 10,000 observations, linear regression likely enabled me to analyze and model the patterns within this dataset.
- By fitting a linear equation to the training data, the model can make predictions on the test data, offering insights into the relationships between different features and the target variable.
- I used gradient descent method to minimize the sum of squared errors and for finding the coefficients of the algorithm.

I used techniques like Standardization, Normalization of the given data:

Standardization:

- Standardization scales the features to have a **mean of 0** and a **standard deviation of 1.**
- Formula: **X_std = (X - mean(X)) / std(X),**

where X_std is the standardized value, X is the original value, mean(X) is the mean, and std(X) is the standard deviation.

Normalization:

- Normalization scales the features to a specific range, typically between 0 and 1.
- Formula: **X_norm = (X - min(X)) / (max(X) - min(X)),**

where X_norm is the normalized value, X is the original value, min(X) is the minimum value, and max(X) is the maximum value.

I used the following mathematical formulas while implementing the code,

$$J(\vec{W},B) = \frac{1}{2m}\sum(\hat{Y}-Y)^2$$

$$J(\vec{W},B) = \frac{1}{2m}\sum(\hat{Y}-Y)^2 \implies J(\vec{W},B) = \frac{1}{2m}\sum(\vec{W}.\vec{X}+B-\vec{Y})^2$$

Gradient Descent:-

$$\frac{\partial J_{\vec{w},b}}{\partial \vec{W}} = \frac{1}{m}(\vec{X^T}).(\vec{W}.\vec{X}+B-\vec{Y})$$

$$\frac{\partial J_{\vec{w},b}}{\partial B} = \frac{1}{m}\sum(\vec{W}.\vec{X}+B-Y)$$

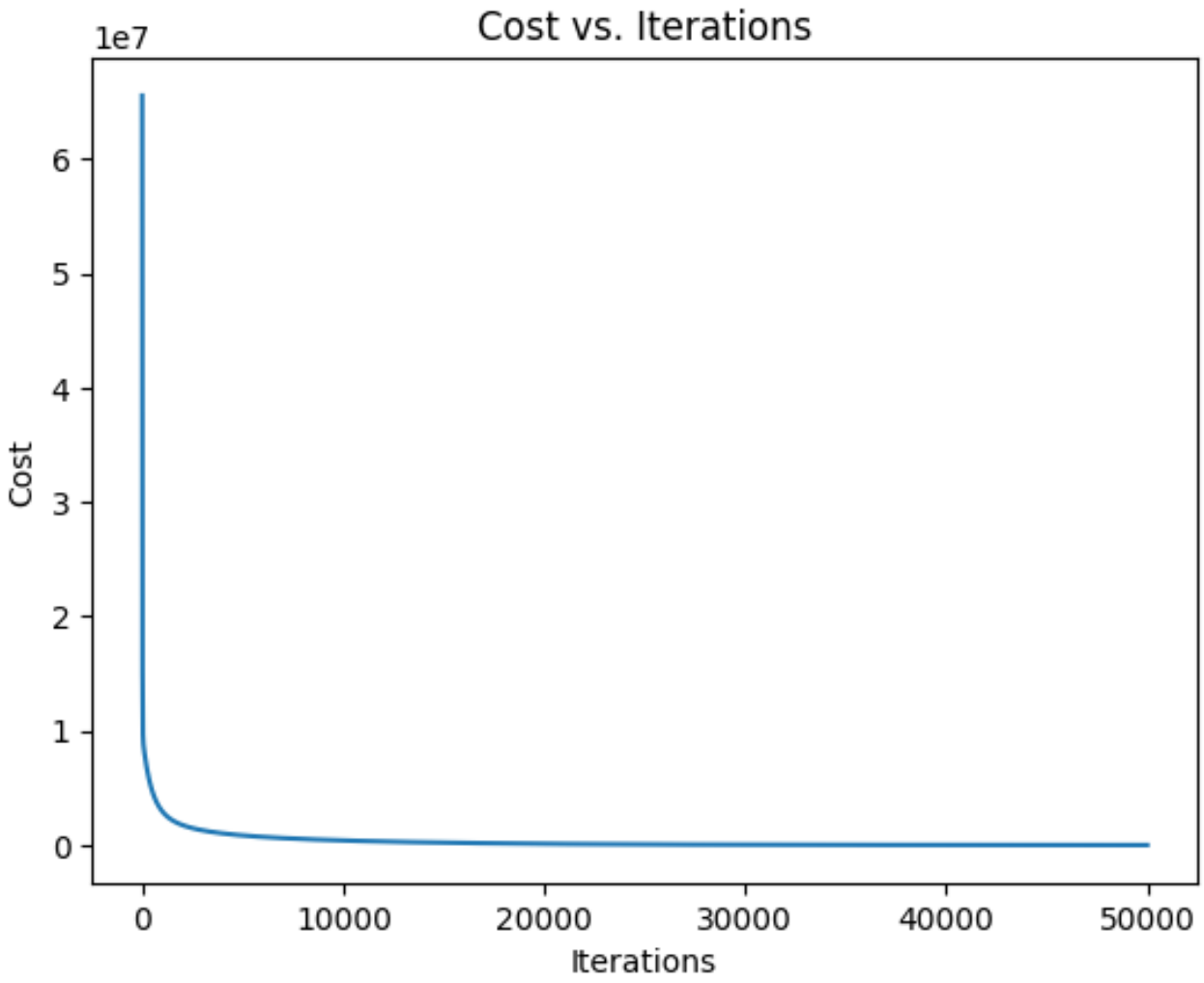
After running my code for various value of iterations ..my cost function reached minimum when I used **learning rate= 0.000001, number of iterations=50000**

Then I used various scores for checking the performance of my model

- Mean Absolute Error
- R2_score

Function to check Performance	Score Obtained
• Mean Absolute Error	96.88569627767112
• **R2_score**	**0.999887551807155**

My cost vs iterations graph obtained as follows:



2. POLYNOMIAL REGRESSION:

- Multi-feature polynomial regression is an Extension of linear regression that accommodates multiple input features, allowing for more complex relationships between the input variables and the target output.
- In this method I used gradient descent method to find the coefficients of polynomial terms and also used gradient descent to minimize sum of squared errors.

After analysis of my train data ....

	target	feature 1	feature 2	feature 3
count	5.000000e+04	50000.000000	50000.000000	50000.000000
mean	1.773548e+06	0.099964	0.012562	-0.205100

std	1.139467e+07	21.842028	4.000527	56.045907
min	-5.353391e+02	-102.142087	-17.236805	-251.346206
25%	1.366923e+02	-14.424551	-2.708389	-37.961040
50%	1.178718e+04	0.265138	0.026296	-0.278907
75%	2.812790e+05	14.773691	2.698501	37.528504
max	8.392773e+08	92.676100	17.647059	232.683184

As there is a observable change in the values of train data ..I had a thought to do feature scaling. But I first tried without any scaling to just check whether my model was performing well or not.

Implementation Details:

- I have created a class "poly_reg():" which takes hyperparameters as inputs, as they are already set by me, and then it trains the model on given train data set.
- Then it makes suitable predictions and calculates metrics to showcase the performance of the model. I have used the following mathematical formulas in my code.

Polynomial Feature Creation:

- Formula: d = a * b * c

Where, a= x1^i,

b=x2^j,

c=x3^k

- Explanation: This creates a new feature by multiplying existing features a, b, and c, resulting in a polynomial term.
- I applied nested loop condition, having 3 loops in a condition. This allowed me to calculate every possible degree term. Later in this code I applied an if condition for necessary adjustments.

Cost Function:

- Formula: **cost = 1/(2*m) * np.sum(np.square(y_pred - y))**
- Explanation: This measures the difference between the predicted values (y_pred) and the actual values (y), indicating how well the model fits the data.

Gradient Descent Update:

- Formula: *theta = theta - learning rate * dtheta*
- Explanation: This iteratively adjusts the model's parameters (theta) to minimize the cost function, finding the best fit for the data.

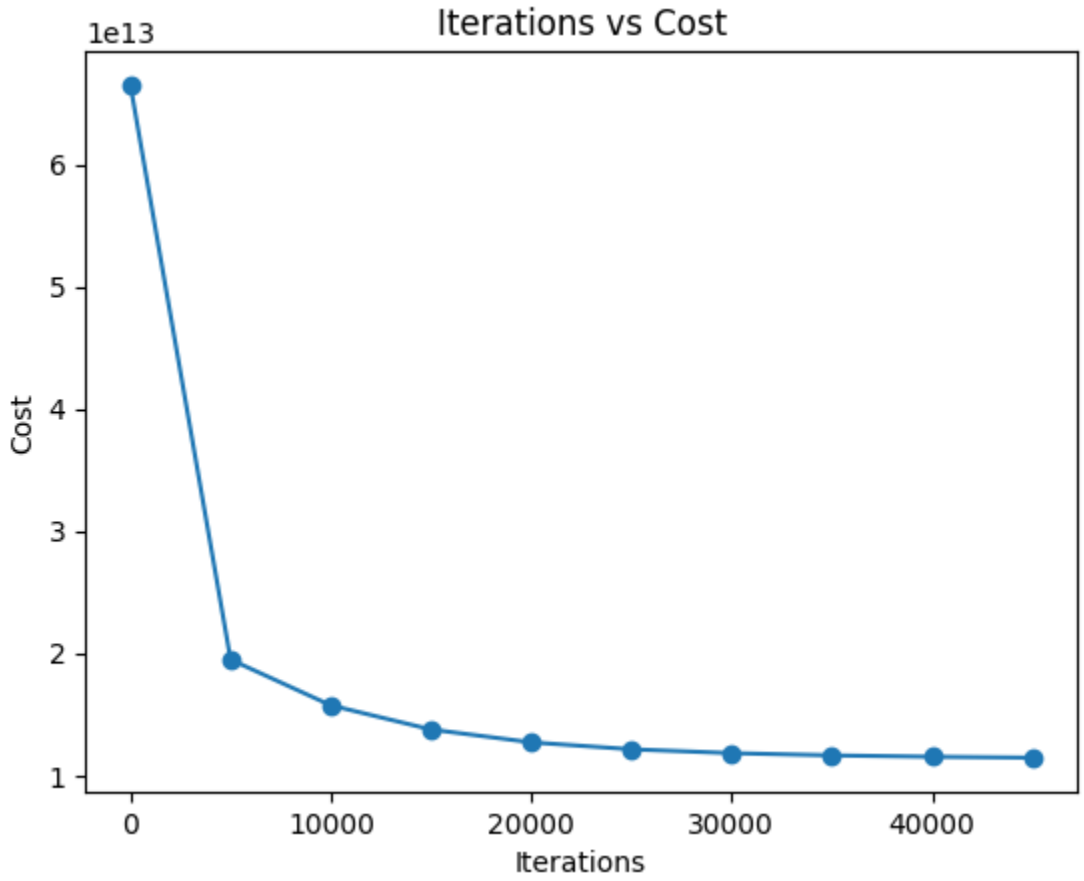
- I had implemented polynomial regression to model the relationship between the input features and the target variable. Initially, I loaded the training and test datasets and converted them into NumPy arrays.
- For the training set, I extracted the input features and the target variable. To capture complex relationships, I extended the features by creating polynomial terms up to the fifth degree. This resulted in a matrix X with various polynomial features.
- The core of the algorithm involved iterating through the data to find the optimal parameters, θ, using gradient descent. In each iteration, predictions were made using the polynomial regression hypothesis, and the cost was computed by comparing these predictions with the actual target values.

- The cost was minimized by adjusting θ through gradient descent, aiming to reduce the difference between predicted and actual values. This process was repeated iteratively until convergence.

- Finally, to validate the model's performance, I applied it to the test dataset. The predictions on the test set were obtained by utilizing the learned θ values and the polynomial features generated in a similar manner as for the training set

- I applied gradient descent to find the optimal coefficients (θ) for our polynomial regression model. The cost function was minimized over 50,000 iterations, and the learning rate was set to 0.00000000000000003 for precision.

COST vs ITERATIONS:



Iterations	COST
0	66490664333888.97
5000	19485549363234.61
10000	15739885956233.814
15000	13764887566112.062
20000	12713794530773.016
25000	12145515087590.846
30000	11830048563191.484
35000	11647359699395.938
40000	11534749598314.146
45000	11459436588704.521

Performance of my model:

I used R2 score metric for checking model's performance.

R2 score	0.9691393135468336

3. LOGISTIC REGRESSION:

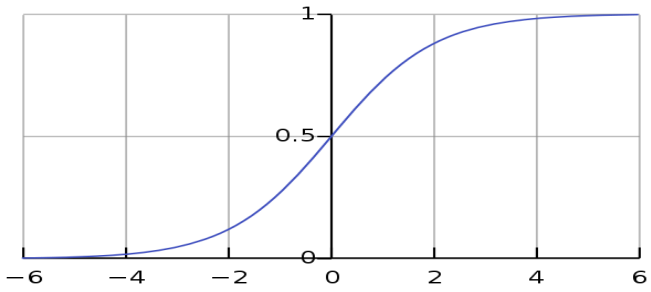
Logistic regression is a statistical method used for binary classification tasks, where the goal is to predict the probability of an event occurring.

In other words, Logistic regression is an ML algorithm, that is used to make predictions for classification problems based on one or more variables.

I used gradient descent method to minimize my mean squared error.

Model Training Function:

- A sigmoid function was defined as sigmoid(x) = 1 / (1 + exp(-x)).



- A model training function was implemented using gradient descent to optimize weights and bias.
- The logistic cost function was utilized for training, given by:

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m}(1-y^{(i)})\log(1-h_\theta(x^{(i)})) + y^{(i)}\log h_\theta(x^{(i)})\right]$$

- The gradients for weights (dW) and bias (dB) were computed.

One-vs-All Training:(one hot encoding)

- A one-vs-all approach was adopted for multi-class classification.
- Models were trained for each class separately using binary label conversion: binary_Y=np.where(Y==class_label,1,0)

Prediction Functions:

- A prediction function for a single model was defined using the sigmoid function.
- Predictions for all models were obtained using a one-vs-all approach.
- The class with the highest predicted probability was selected as the final prediction.

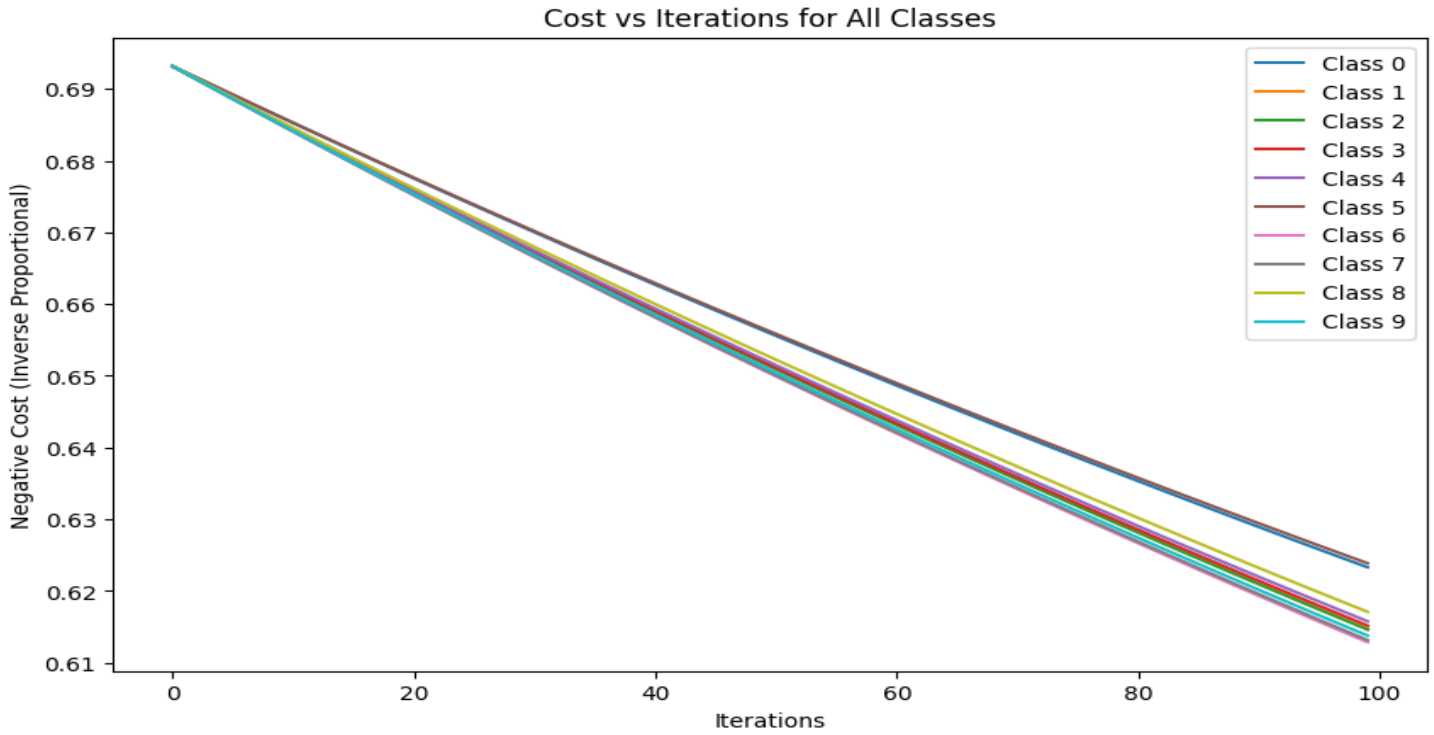
Training Loop:

- The model training loop iterated for a specified number of iterations, updating weights and bias at each step.
- Cost values were tracked for analysis and displayed every 10% of the iterations.

My model' cost vs iterations analysis:

Iterations	Cost
0	0.6931471805599435
10	0.6840240129104036
20	0.6751462018873691
30	0.6665068595284565
40	0.658099227142128
50	0.6499166799922947
60	0.6419527311947568
70	0.634201034875399
80	0.6266553886403052
90	0.6193097354076829

As I have created 10 classes in my code. My iterations vs cost graph is as follows:



My model's accuracy is as follows:

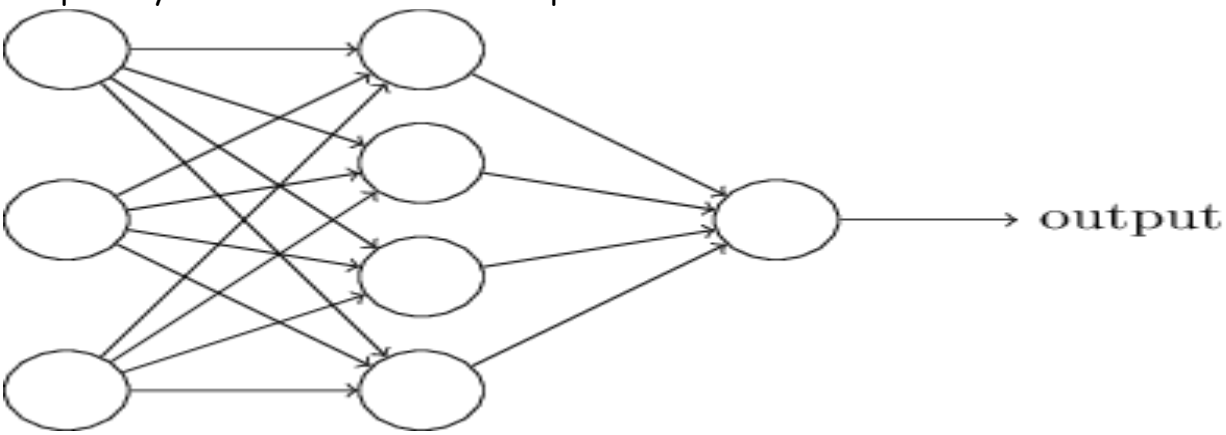
accuracy is calculated by taking the mean of an array where each element indicates whether the predicted label matches the true label.

Accuracy on training set: 82.24000000000001 %

4. N-LAYER NEURAL NETWORK:

Neural networks are a class of machine learning models inspired by the structure and functioning of the human brain. Neural networks consist of interconnected nodes, also known as neurons or units, organized into layers.

- Input Layer: Receives input data.
- Hidden Layers: Process the input through a series of weighted connections and apply activation functions to produce output.
- Output Layer: Produces the final output.
-



Implementation details:

Read and Converted Data:

- **I read the training data from the file "Classification_train.csv" and converted it into a NumPy array.**

Shuffled Data:

- **I shuffled the data randomly to introduce variability and avoid any order-related bias.**

Split Development and Training Sets:

- **I extracted the first 1000 rows for development data and the remaining rows for training data.**

Preprocessed Data:

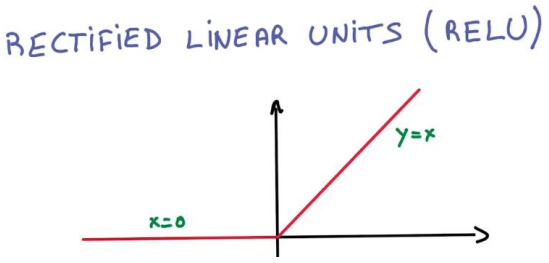
- **For both development and training sets, I extracted labels and features and normalized the pixel values to a range of 0 to 1. [NORMALISATION]**

Initialized Neural Network Parameters:

- **I had initialized the parameters for the neural network, including weights and biases for two hidden layers.**
- **The hidden layer has 10 neurons.**
- The input layer has 784 neurons, representing the features of the dataset.
- The output layer has 10 neurons, corresponding to the possible classes or categories.

Implemented Activation Functions:

- **I implemented ReLU (Rectified Linear Unit) and SoftMax activation functions for the hidden layers and output layer, respectively.**



Implemented Forward Propagation:

- **I implemented the forward propagation step to compute the output of the neural network for a given input.**

One-Hot Encoded Target Variable:

- **I one-hot encoded the target variable (labels) to facilitate the training process.**

Implemented Backward Propagation:

- **I implemented the backward propagation step to calculate the gradients of the parameters with respect to the loss.**

Updated Parameters Using Gradient Descent:

- **I updated the parameters (weights and biases) using gradient descent to minimize the training loss.**

Trained Neural Network:

- **I trained the neural network using gradient descent over ***500 iterations*** with a ***learning rate of 0.10*.**

Made Predictions on Training Examples:

- **I made predictions on specific examples from the training set to evaluate the network's performance.**

Tested Predictions on Development Set:

- **I tested the predictions on the development set and calculated the accuracy of the model.**

Displayed Predictions and Labels:

- **For a few examples from the training set, I displayed the model's predictions alongside the actual labels.**

Made Predictions on Test Data:

- I loaded the test data from the file "Classification_test.csv," preprocessed it similarly to the training data, and used the trained neural network parameters to make predictions.

Displayed Test Predictions:

- **I displayed the predictions made by the neural network for the test data.**
- Accuracy was determined by assessing how well the model's predictions aligned with the true labels in my dataset, and this process was repeated at various iterations during the training phase.

Iterations	Accuracy
0	0.0686
50	0.2950
100	0.5519
150	0.7136
200	0.7860
250	0.8266
300	0.8517
350	0.8696
400	0.8821
450	0.8907

490	**0.8969**

5. K-NEAREST NEIGHBOURS [KNN] :

K-Nearest Neighbors (KNN) is a simple algorithm for classification and regression, predicting outcomes based on the majority class or average value of k-nearest neighbors in the feature space using a distance metric (usually Euclidean).

In the process of implementing the K-Nearest Neighbors (KNN) algorithm, I first focused on determining the optimal value for the parameter 'k,' which represents the number of neighbors considered for predictions.

- I implemented the KNN algorithm from scratch in Python using NumPy and Pandas.

- I created a KNN class with the capability to accept a parameter 'k' (default set to 4). The choice of 'k' significantly impacts the algorithm's performance.

- I applied the KNN algorithm to a given training dataset.

- By loading the training data and splitting it into training and test sets, I systematically experimented with different values of 'k.' For this, I varied 'k' and observed the algorithm's accuracy on the test set.
- I evaluated the performance of the KNN algorithm for different 'k' values.
- After rigorous experimentation, I discovered that setting 'k' to 4 yielded optimal performance in terms of accuracy on the test data.
- Having determined the optimal 'k' value, I proceeded to implement the KNN algorithm with 'k=4' for predicting labels on the test data.
- The implementation involved computing distances, finding the k-nearest neighbors, and employing a majority voting mechanism to determine the predicted label.
- I executed the optimized KNN algorithm on the test data.
- The code executed successfully, predicting labels for the test data. The results, including accuracy and runtime, were printed for further analysis.
- I loaded the new data, extracted features, and printed the predicted labels based on the trained KNN model.
- The implemented KNN algorithm demonstrated its efficacy, achieving accurate predictions on the test data with the chosen 'k' value of 4.

When I checked accuracy on my train data (as I split the train data into 2halves where 1st half is for train data. And remaining for test data), I came to know that accuracy as:

Accuracy: 0.9718666666666667

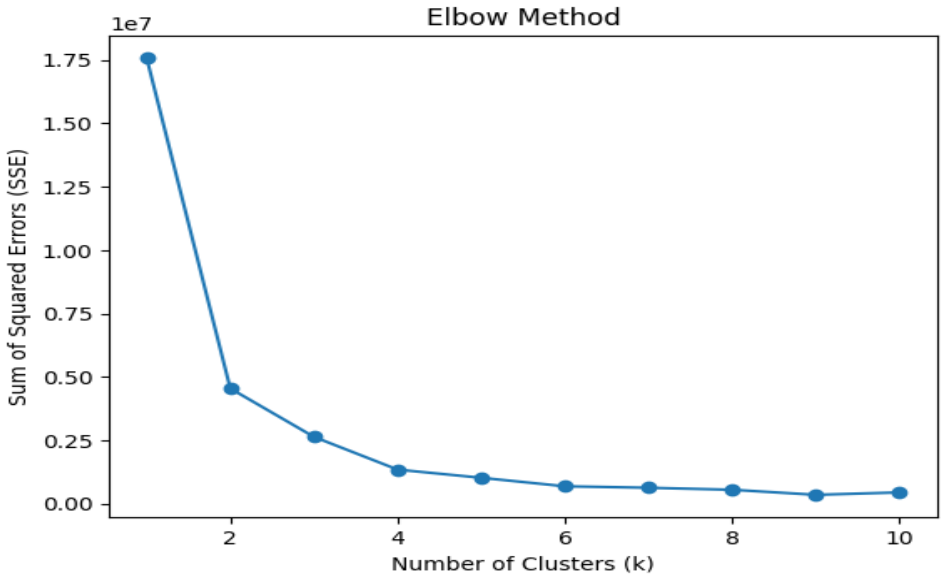
6.K-means Clustering:

K-Means clustering is an unsupervised machine learning algorithm used for partitioning a dataset into K distinct, non-overlapping subsets (clusters). It works by iteratively assigning data points to the nearest cluster centroid and updating the centroids based on the mean of the assigned points. The process continues until convergence, where the centroids no longer change significantly. The algorithm aims to minimize the sum of squared distances between data points and their assigned cluster centroids, effectively grouping similar data points together.

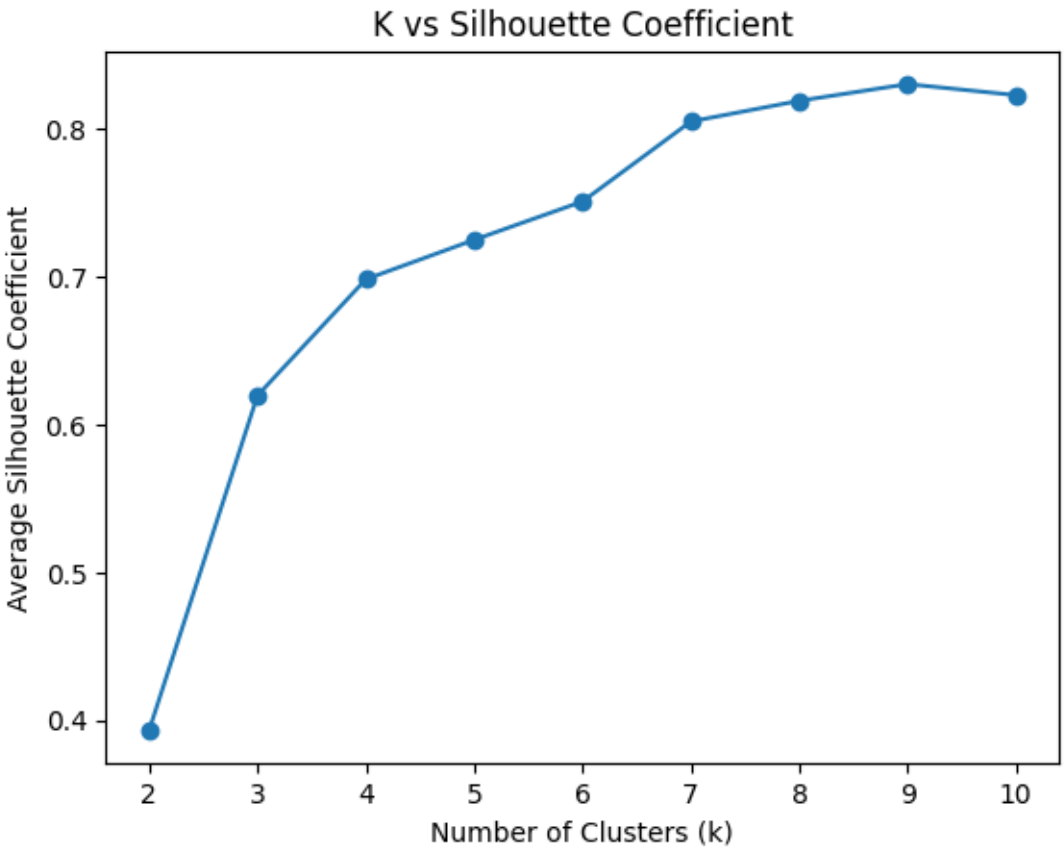
Implementation details:

- Features are extracted from the dataset, stored in variable X.
- Euclidean distance is calculated using the function **`euclidean_distance(x1, x2)`**
- Centroids are initialized randomly using the function **`initialize_centroids(X, k)`**.
- The main K-Means algorithm is implemented in the function **`k_means(X, k, max_iters=100)`**.
- Centroids are updated iteratively
- To determine the optimal number of clusters, the elbow method is used

- The Sum of Squared Errors (SSE) is calculated for different cluster numbers.
- Results are visualized in the 'Elbow Method' plot.
- The optimal number of clusters is determined using the elbow method.
- In this specific case, **`optimal_k`** is set to 3.



- The silhouette coefficient is calculated using the function **`silhouette_coefficient(data, labels, centroids)`**.
- It evaluates the quality of clustering by measuring how well-separated clusters are.
- Another implementation of K-Means clustering is provided with additional functionalities.
- A function for calculating the silhouette coefficient and visualizing clustering results is included.
- The number of clusters, **`k`**, is set to 9 in this case.



The silhouette coefficient typically ranges from -1 to 1, and its interpretation is as follows:

Near +1: It indicates that the sample is far away from the neighboring clusters. This is a good indication of a well-defined cluster.

Near 0: It indicates that the sample is on or very close to the decision boundary between two neighboring clusters. This could imply overlapping clusters.

I got silhouette value as: 0.8378260229866437(k=9)

0.5037478693897947(k=3)